HPC System Architecture

Ivy Peng

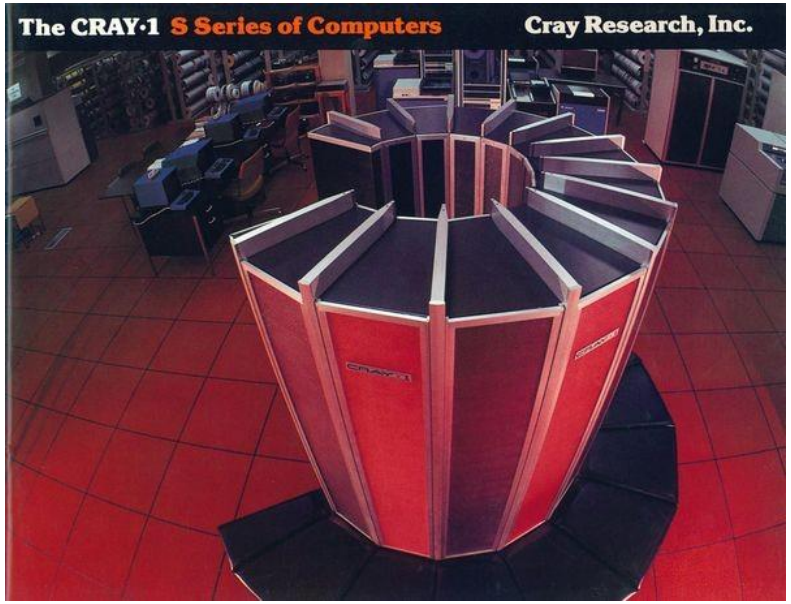
Associate Professor

Outline

- History and Recent Development in HPC systems
- HPC and Supercomputer Architecture
- Resource Management and Job Scheduling
- Scalability

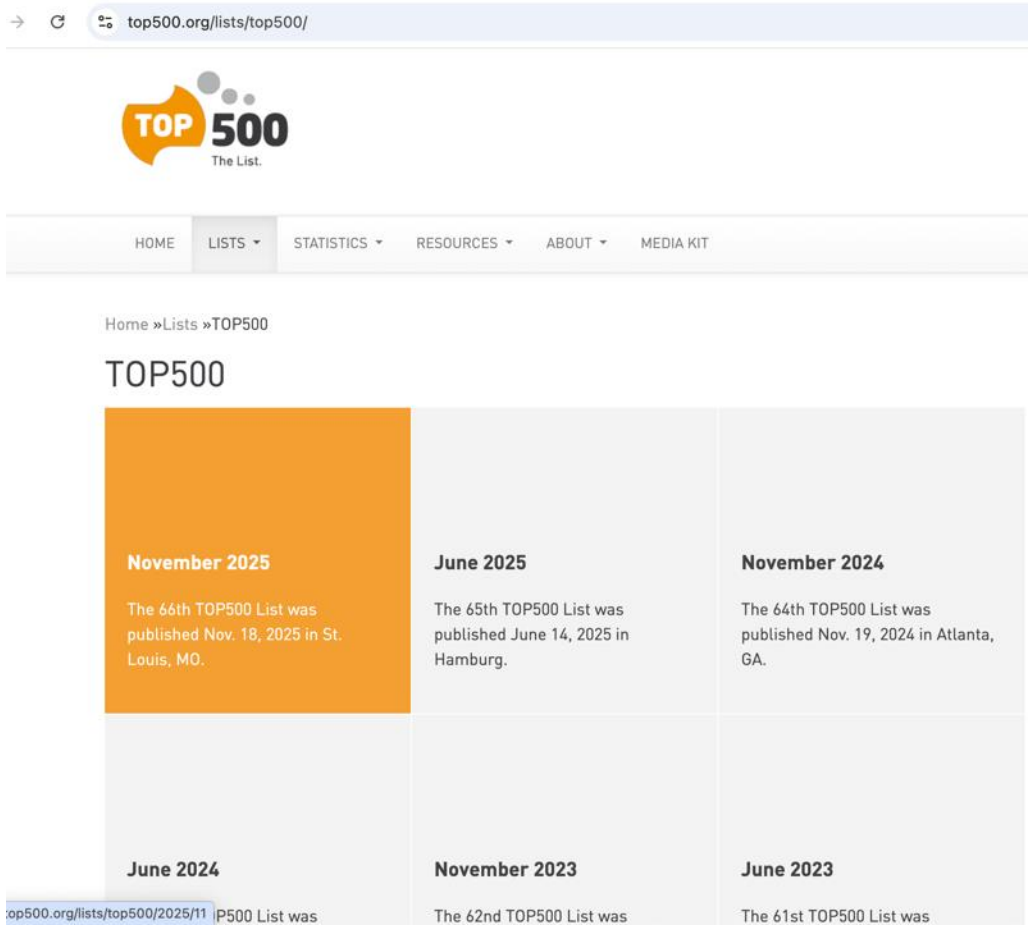
The history of High-performance Computing (HPC)

- 1960s — Birth of Supercomputing: First machines designed specifically for scientific computing
- 1970s — Vector Supercomputers: Cray-1
- 1990s — Massively Parallel Systems: Shift to parallel computing
- 2010s — Petascale Computing: GPU becomes common
- 2020s — Exascale Era: Systems exceed 1 exaflop (10^{18} operations/sec). Summit, Frontier supercomputers



HPC systems and Supercomputers

- Twice per year, supercomputers in the world are ranked to update the Top500 List. https://top500.org/
  - Rapid changes in the high-end computing technologies: Compute, Memory, Network
  - Ranking is done via running a set of benchmarks to obtain performance
    - Top500 ranking uses the **Linpack** benchmark (https://top500.org/project/linpack/), essentially a dense linear system, to obtain floating point operations per second (computing throughput)
    - Other ranking like HPCG and Green500 use different **benchmarks**

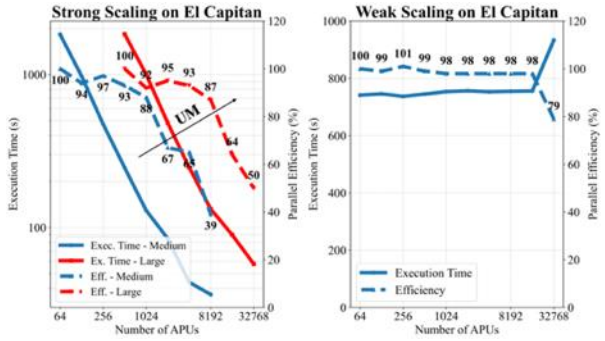


The Latest Top500 Ranking

Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	**El Capitan** - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOE/NNSA/LLNL United States	11,340,000	1,809.00	2,821.10	29,685
2	**Frontier** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE DOE/SC/Oak Ridge National Laboratory United States	9,066,176	1,353.00	2,055.72	24,607
3	**Aurora** - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128	1,012.00	1,980.01	38,698
4	**JUPITER Booster** - BullSequana XH3000, GH Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux, EVIDEN EuroHPC/FZJ Germany	4,801,344	1,000.00	1,226.28	15,794
5	**Eagle** - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR, Microsoft Azure Microsoft Azure United States	2,073,600	561.20	846.84	
6	**HPC6** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X,	3,143,520	477.90	606.97	8,461

Source: https://asc.llnl.gov/exascale/el-capitan

- El Captitan supercomputer is the No. 1 ranked today.
- Our plasma simulation code iPIC3D can scale on 32000 GPUs on El Captitan with high parallel efficiency/scalability



Outline

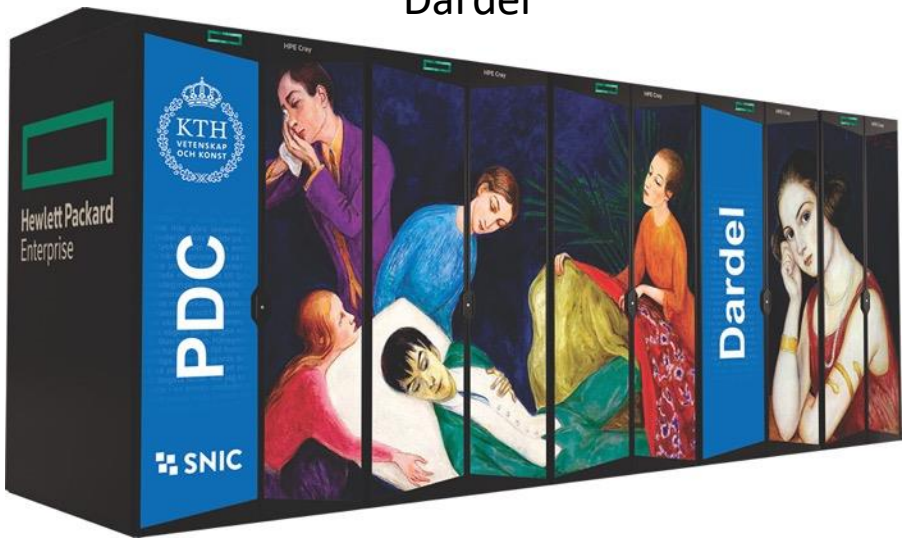
- History and Recent Development in HPC systems
- **HPC and Supercomputer Architecture**
- Resource Management and Job Scheduling
- Scalability

An Overview of HPC Architecture

El Capitan

Source: https://www.nextplatform.com/2023/07/10/lining-up-the-el-capitan-supercomputer-against-the-ai-upstarts/

Dardel

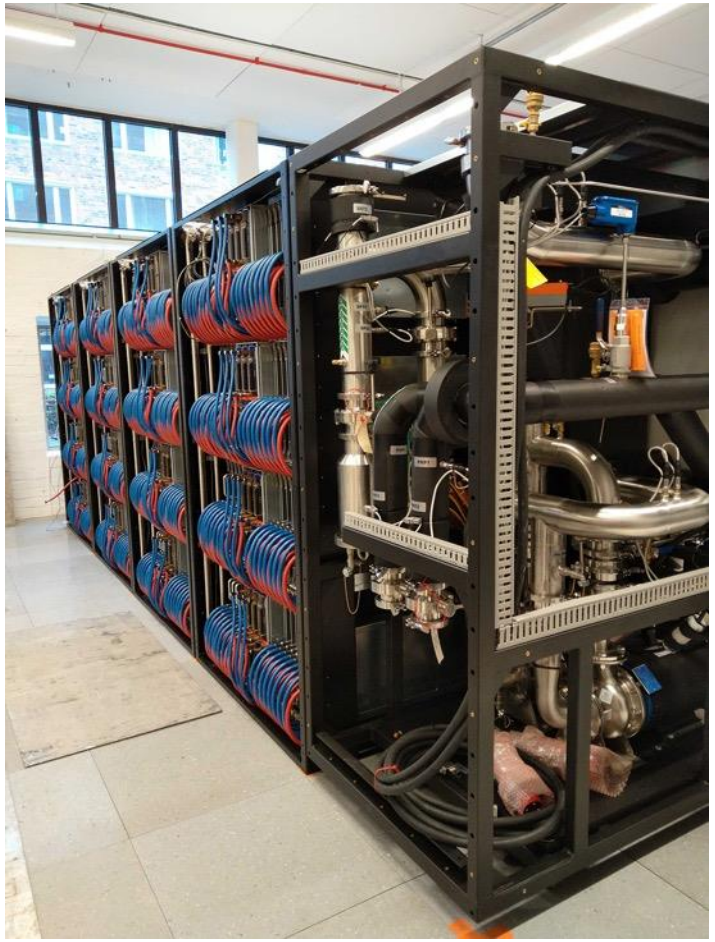


https://www.pdc.kth.se

An Overview of HPC Architecture

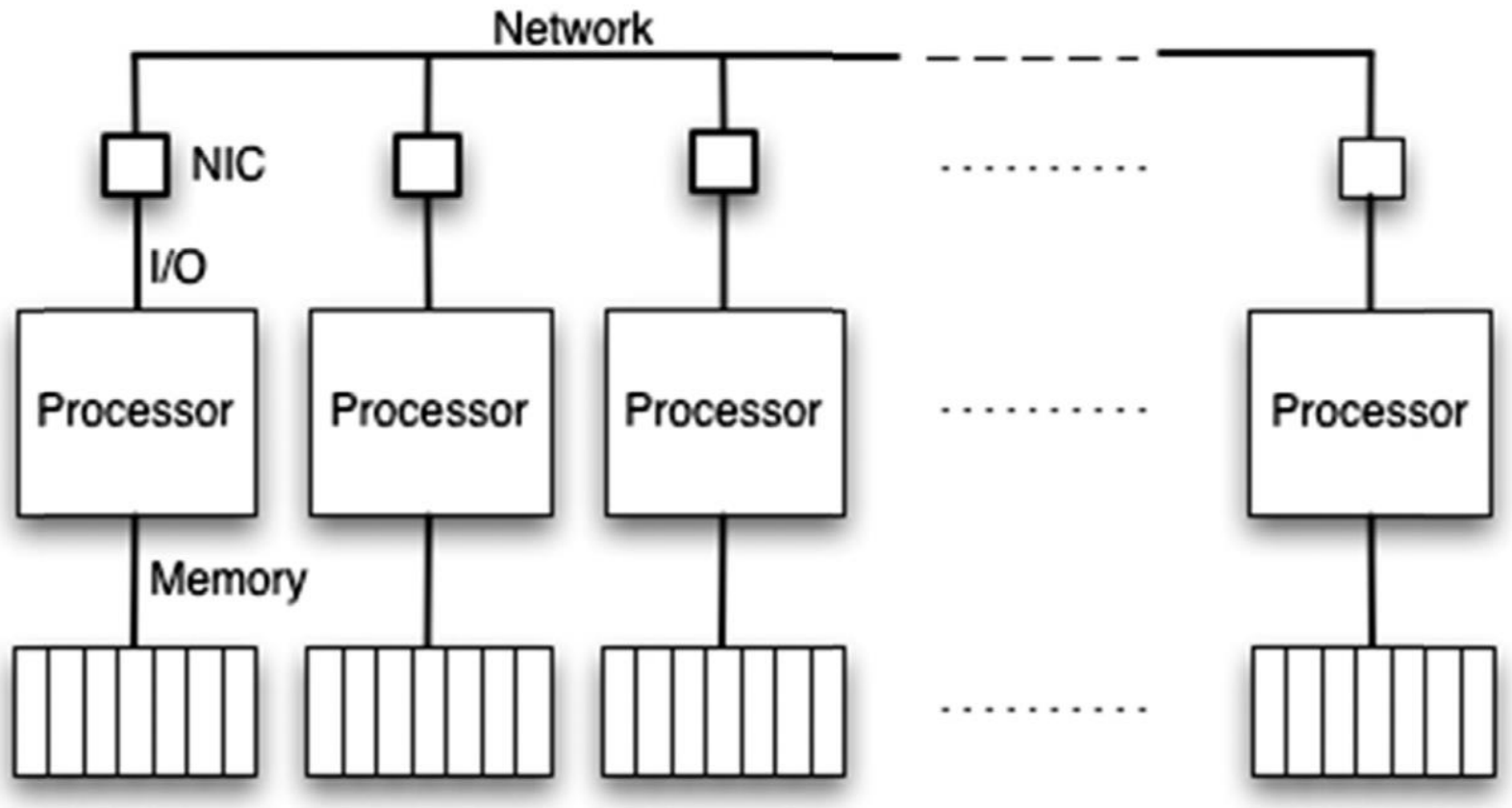
El Capitan

Dardel



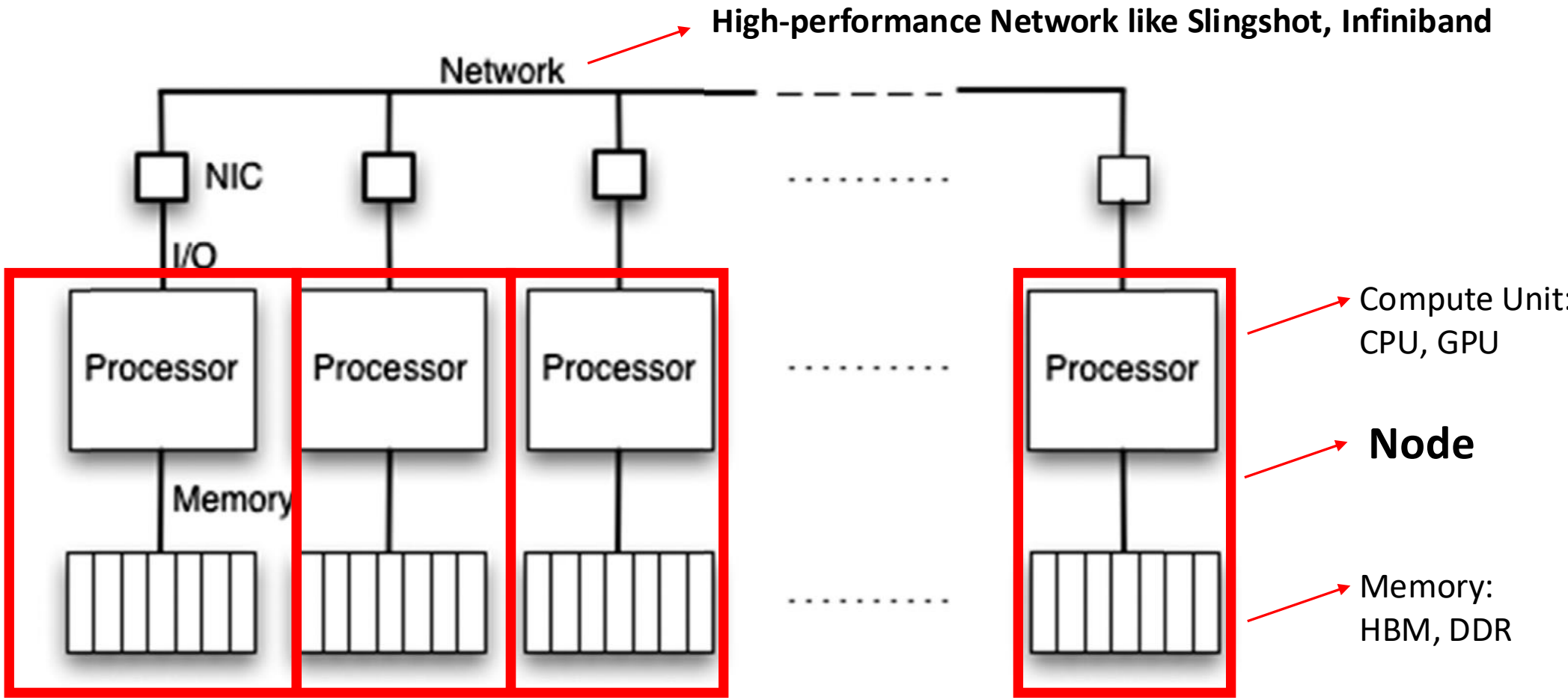
An interesting blog on the installation of Dardel supercomputer can be found at https://www.pdc.kth.se/hpc-services/computing-systems/dardel-hpc-system/dardel-photo-timeline/phase-1-of-dardel-arrives-at-pdc-1.1091589

An Abstraction of HPC Architecture



Source: Textbook; High Performance Computing Modern Systems and Practices by Thomas Sterling, Matthew Anderson, and Maciej Brodowic

An Abstraction of HPC Architecture



Source: Textbook; High Performance Computing Modern Systems and Practices by Thomas Sterling, Matthew Anderson, and Maciej Brodowic

HPC Architecture

- 1 or multiple **Rows**:
  - 4 rows in this picture
- Each row has multiple **cabinet / rack**:
  - 10 racks in this Dardel picture
- Each rack / cabinet consists of many **blades** (enclosure)
- Each blade enclosure may have 1-8 **nodes**

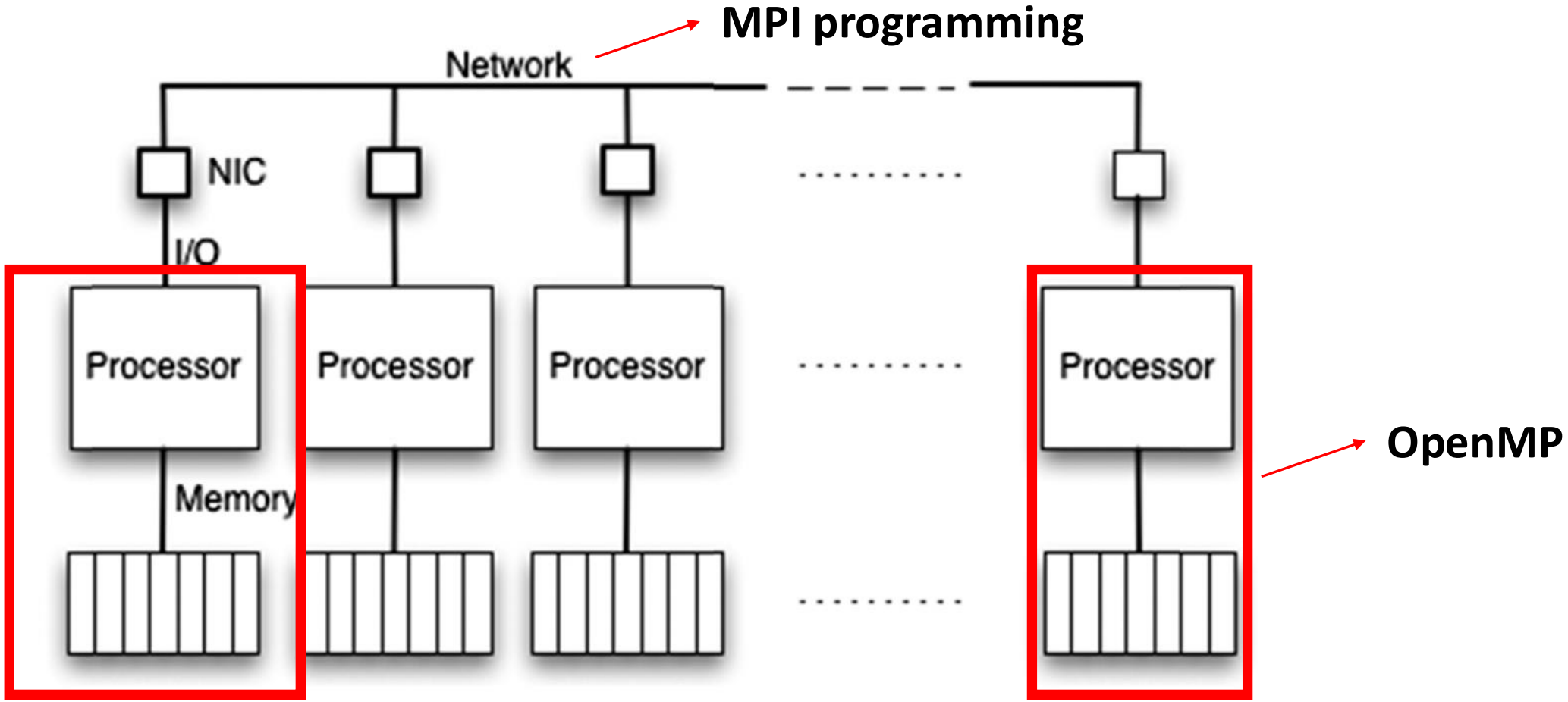
A blade of two nodes

El Capitan

Dardel

Installation of blades on Dardel

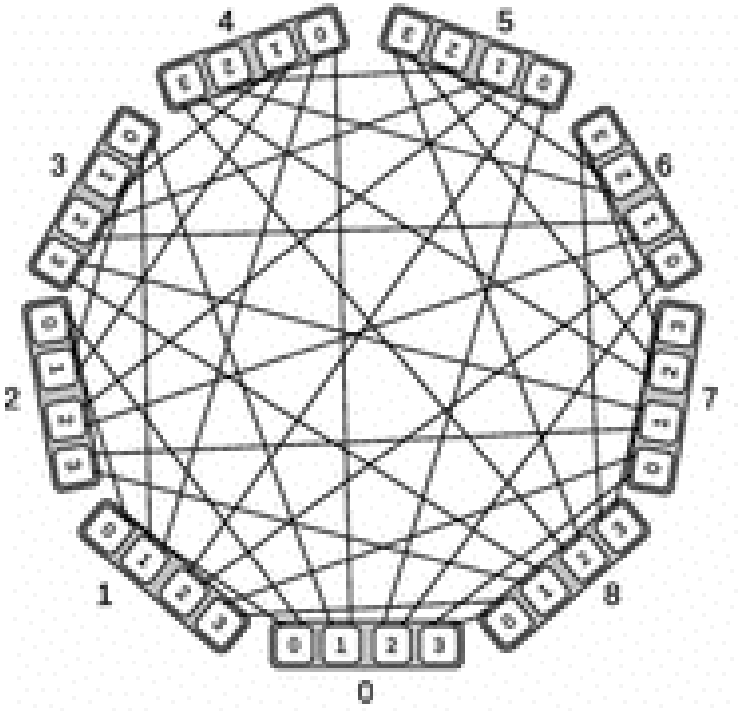
An Abstraction of HPC Architecture

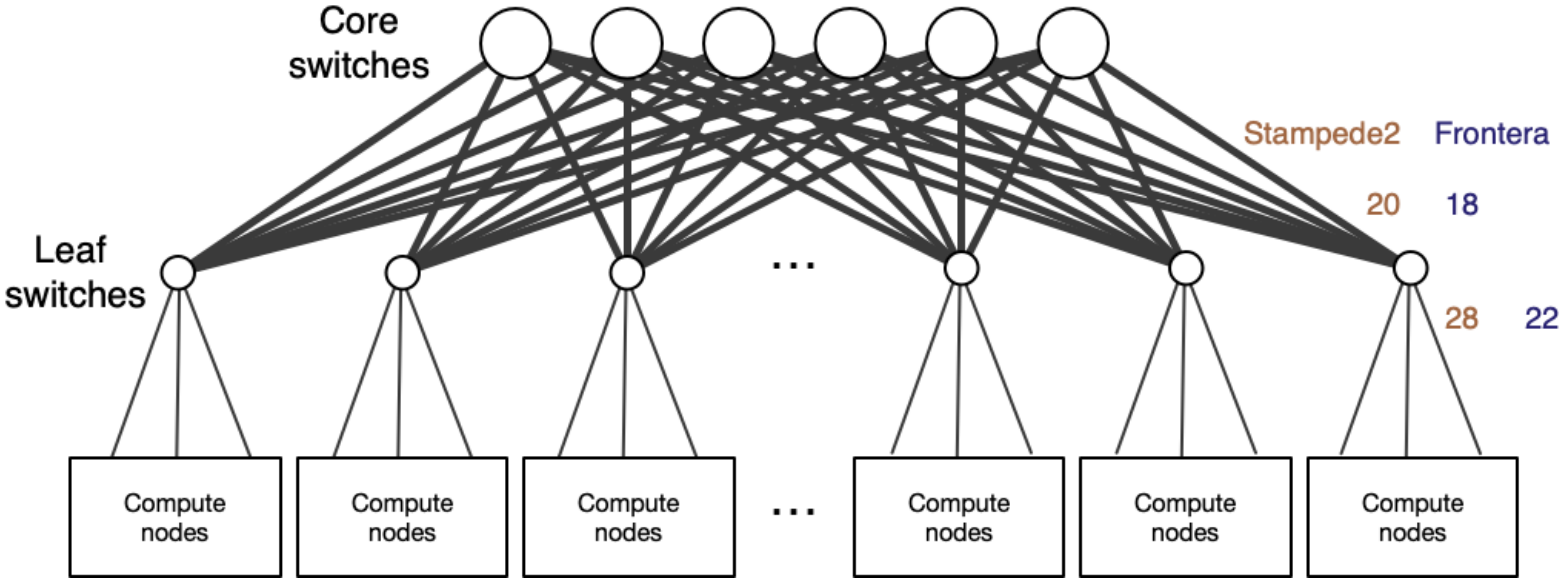


Source: Textbook; High Performance Computing Modern Systems and Practices by Thomas Sterling, Matthew Anderson, and Maciej Brodowic

HPC Network Topology

- Two big categories: Fat-tree, Dragonfly

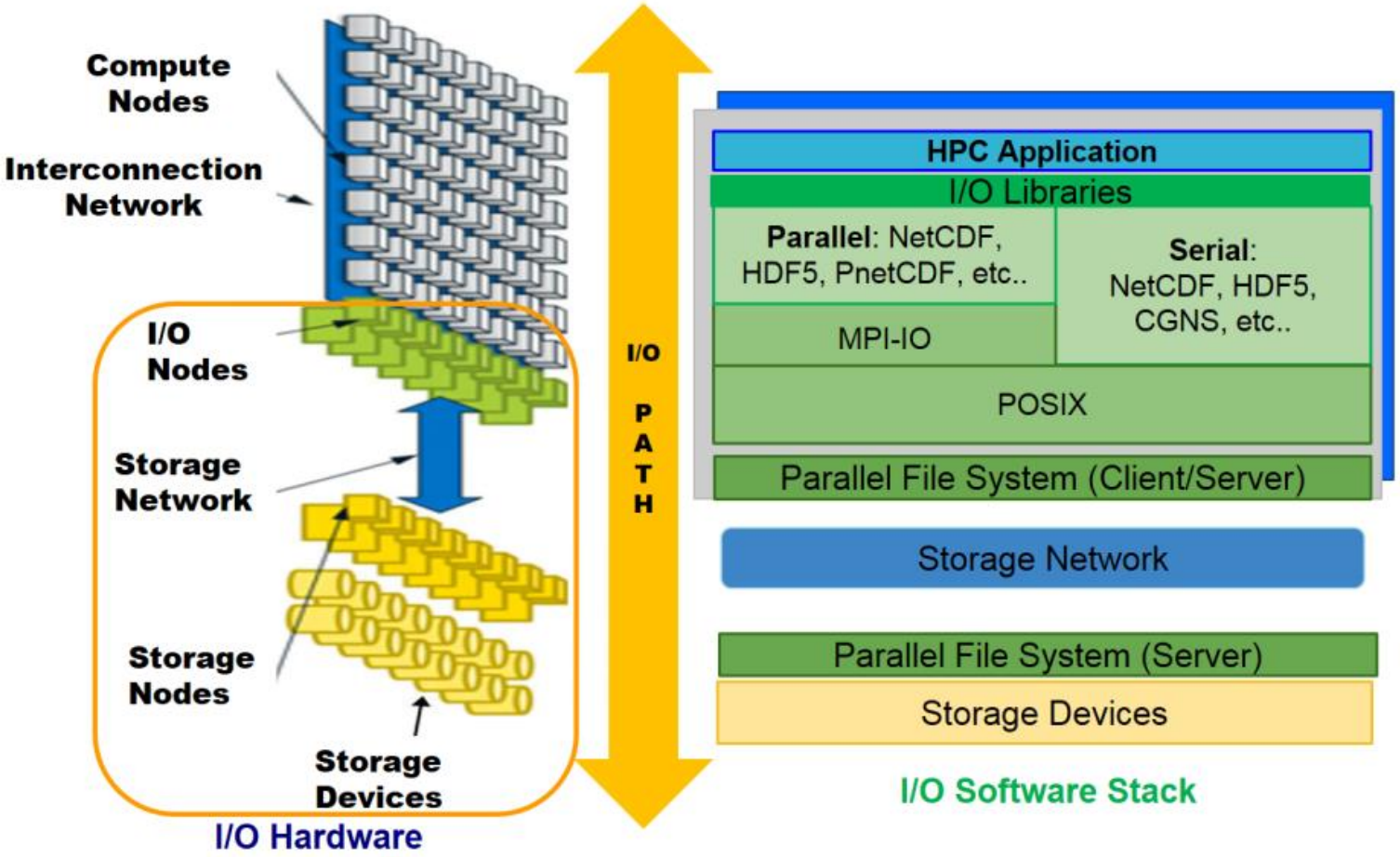




Source: https://cvw.cac.cornell.edu/parallel/hpc/clusters

HPC Storage Parallel

- Massive data size from HPC applications: 10~TB data per simulation
- Parallel I/O, Parallel file system: Lustre filesystem is most used one

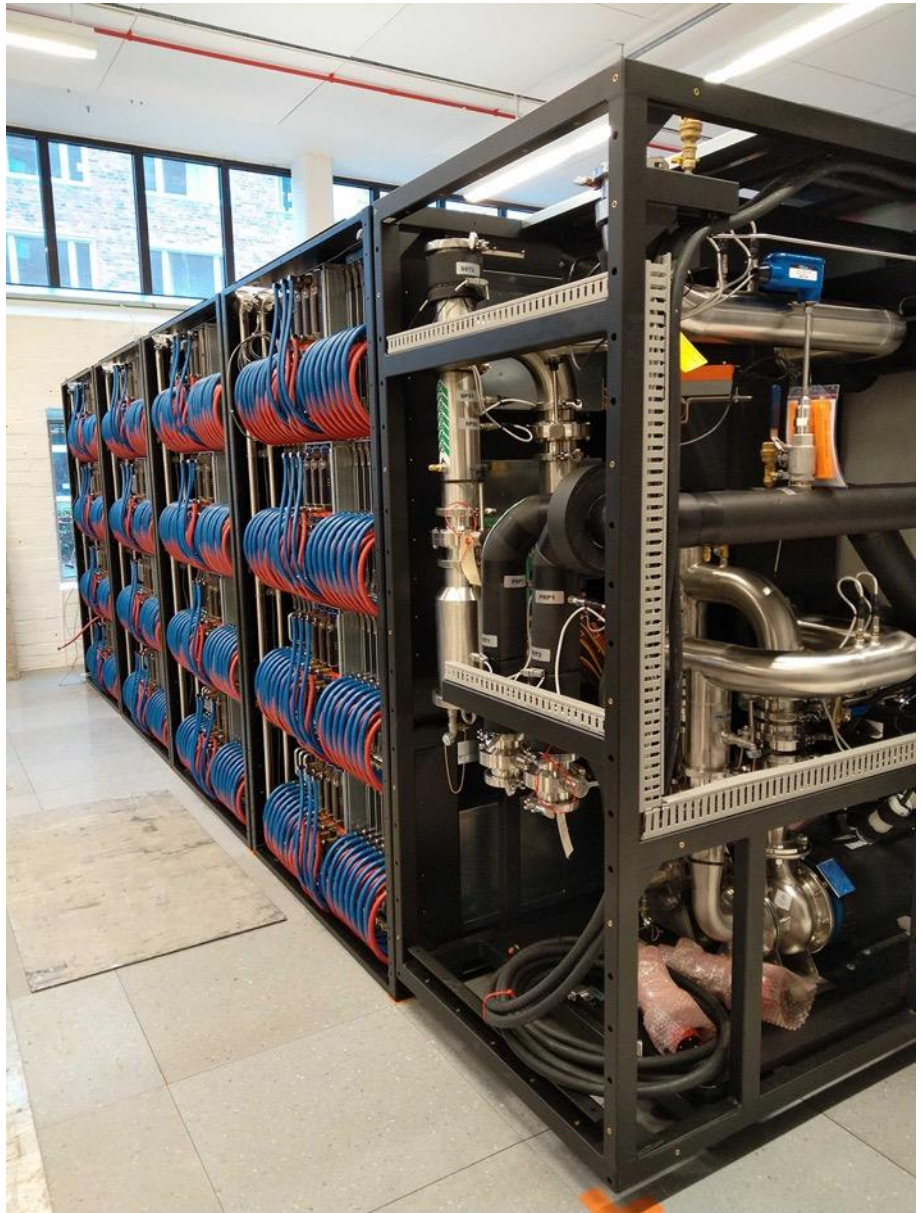


Source: https://www.nhr.kit.edu/userdocs/horeka/parallel_IO/

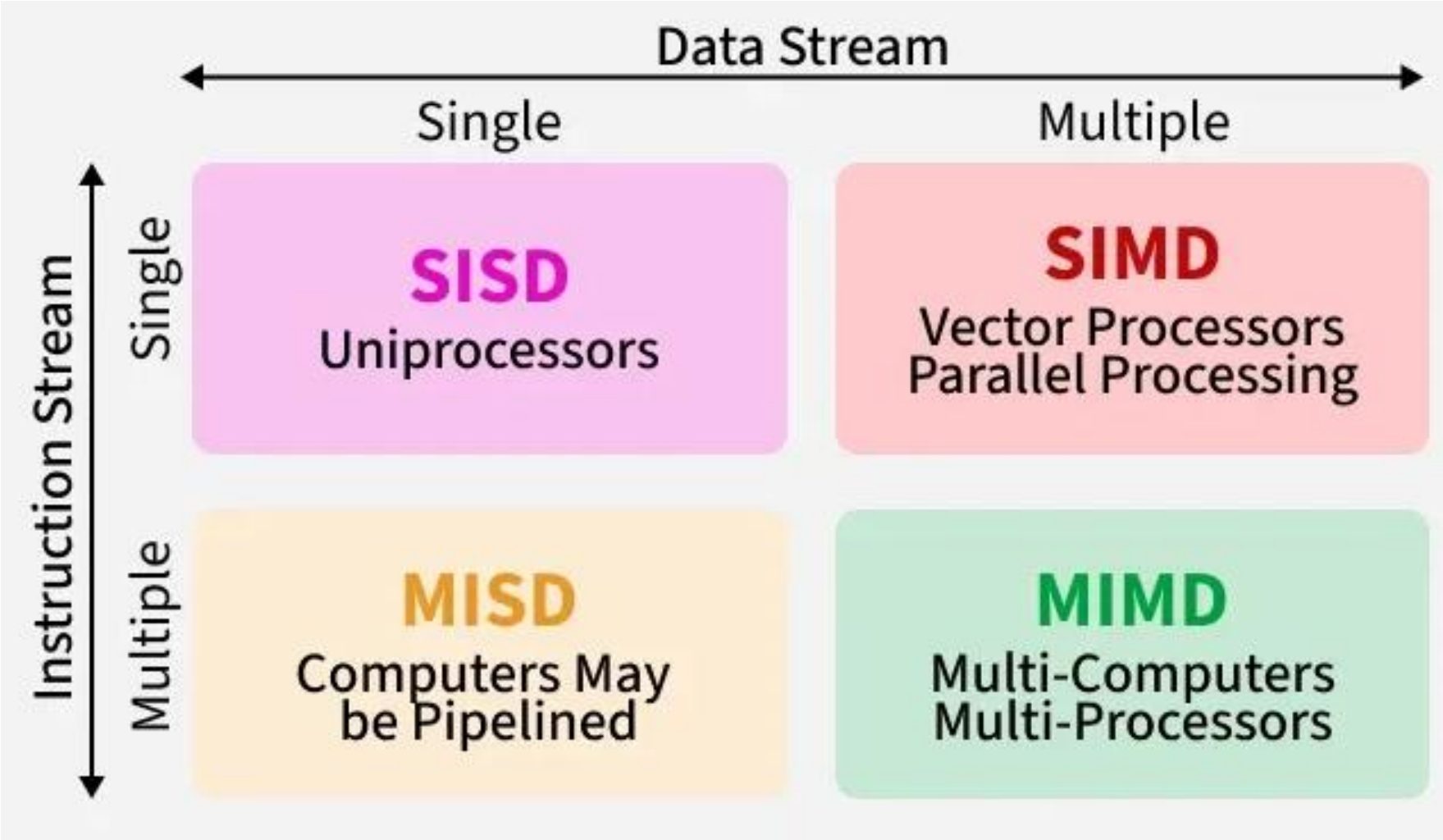
Power Management and Cooling becomes challenging

- Today, a high-end CPU only processor like Intel Sapphire Rapids system can reach 900 Watt on dual-socket
- A single GPU like Nvidia H100 can reach peak 700 Watt power consumption
- A supercomputer has thousands nodes
- Cooling becomes challenging
- High power consumption does not mean low energy-efficiency if a computation task can be completed in a shorter time
  - Power Efficiency = $\frac{\text{Computing Throughput}}{\text{Power Consumption}}$

    = FLOPs per Watt



Flynn's Taxonomy



Which category do today's HPC systems belong to ?

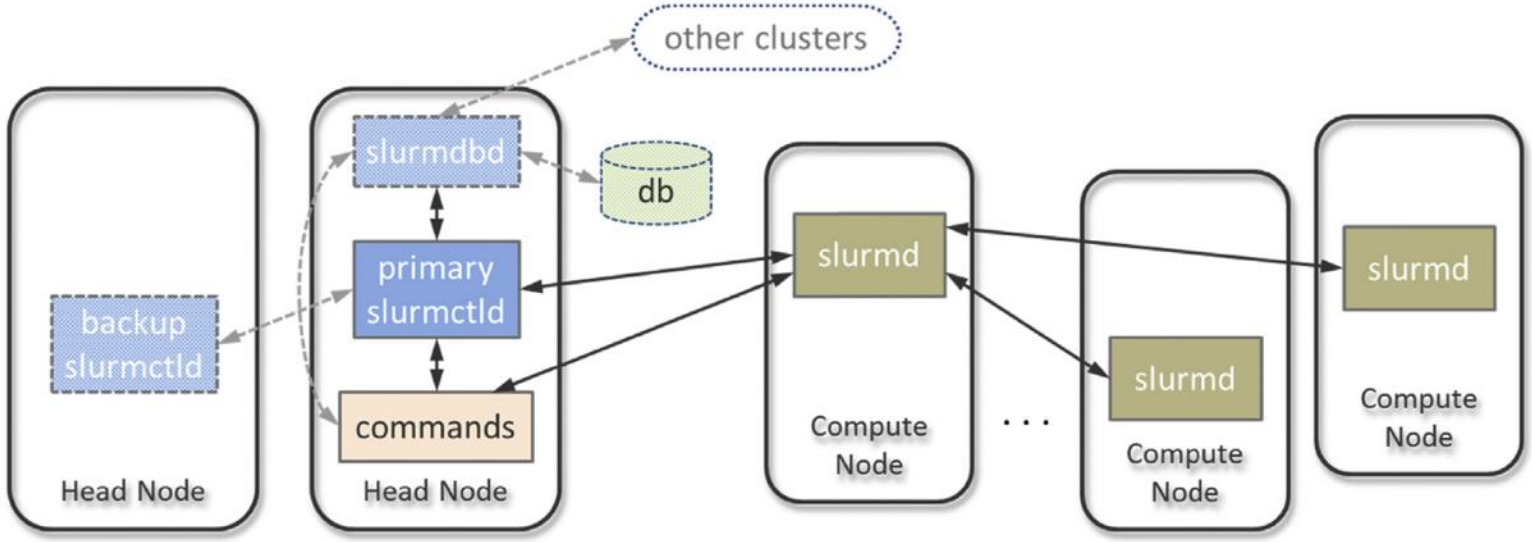
Outline

- History and Recent Development in HPC systems
- HPC and Supercomputer Architecture
- **Resource Management and Job Scheduling**
- Scalability

Resource Management and Job Scheduling

- Resource management tools are a part of the HPC software stack.
- Popular software includes: **Slurm**, Moab, Flux
- Main resource types
  - Compute **Nodes**: a parallel application can run on N nodes.
  - Processing **Cores per node** (incl. Accelerators): specifying shared or exclusive allocation of nodes to workloads
  - **Memory**
  - Storage

How to choose N for a job ?



Sterling, T., Brodowicz, M., & Anderson, M. Chp5: Simplified architecture of SLURM

Resource Management and Job Scheduling

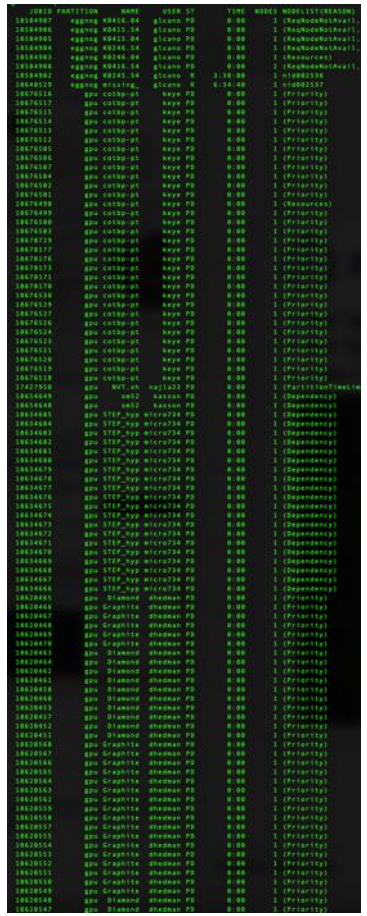
- Three main functions:
  - resource allocation: each job asks for certain resources
  - workload scheduling: pending jobs stay in a queue
  - support distributed workload execution and monitoring, e.g., bootstrapping, pro-diagnosis, …

- Different job scheduling algorithms:
  - Backfill
  - priority-based FIFO scheduling
  - Gang scheduling: time-slicing

```
$ salloc –h
Usage: salloc [OPTIONS(0)...] [ : [OPTIONS(N)]] [command(0) [args(0)...]]


Parallel run options:
  -A, --account=name         charge job to specified account
  -b, --begin=time           defer job until HH:MM MM/DD/YY
      --bell                 ring the terminal bell when the job is allocated
      --bb=<spec>            burst buffer specifications
      --bbf=<file_name>      burst buffer specification file
  -c, --cpus-per-task=ncpus  number of cpus required per task
      --comment=name         arbitrary comment
      --container            Path to OCI container bundle
      --container-id         OCI container ID
      --cpu-freq=min[-max[:gov]] requested cpu frequency (and governor)
      --delay-boot=mins      delay boot for desired node features
  -d, --dependency=type:jobid[:time] defer job until condition on jobid is satisfied
      --deadline=time        remove the job if no ending possible before
                             this deadline (start > (deadline - time[-min]))
  -D, --chdir=path           change working directory
      --get-user-env         used by Moab.  See srun man page.
      --gres=list            required generic resources per node
      --gres-flags=opts      flags related to GRES management
  -H, --hold                 submit job in held state
  -I, --immediate[=secs]     exit if resources not available in "secs"
  -J, --job-name=jobname     name of job
  -k, --no-kill              do not kill job on node failure
  -K, --kill-command[=signal] signal to send terminating job
  -L, --licenses=names       required license, comma separated
  -M, --clusters=names       Comma separated list of clusters to issue
                             commands to.  Default is current cluster.
                             Name of 'all' will submit to run on all clusters.
                             NOTE: SlurmDBD must up.
  -m, --distribution=type    distribution method for processes to nodes
```

Job Scheduling on Real Supercomputers is Busy

- Run “sinfo” on the login node to check the system status
- Run “squeue” to check job queues
- Thus, reduce the number of jobs sent to the queue: **one job per group, submit job script early**

```
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
gpu          up 1-00:00:00      1 drain$ nid002832
gpu          up 1-00:00:00      1  maint nid002833
gpu          up 1-00:00:00      1 drain* nid002807
gpu          up 1-00:00:00      1  drain nid002808
gpu          up 1-00:00:00      6   resv nid[002810-002815]
gpu          up 1-00:00:00     52  alloc nid[002792-002806,002809,002816-002831,002834-00285
main         up 1-00:00:00     18   mix- nid[001013,001015,001020,001022,001024,001027,00103
071,001074]
main         up 1-00:00:00     13 drain$ nid[001036,001090,001194-001195,001222-001223,00130
main         up 1-00:00:00      4  down$ nid[001416-001419]
main         up 1-00:00:00     51  maint nid[001037-001039,001088-001089,001091,001192-00119
303,001336-001338,001352,001354-001355,001388-001391,001445-001447,001844-001846,001848-0018
main         up 1-00:00:00     24    mix nid[001014,001016,001021,001023,001025,001028,00103
063,001066,001070,001072-001073]
main         up 1-00:00:00    876  alloc nid[001012,001017-001019,001026,001029,001031,00103
067,001075-001087,001092-001191,001204-001207,001212-001219,001224-001299,001304-001335,0013
1443,001448-001531,001756-001843,001852-001907,001912-002023,002552-002567,002588-002675,002
main         up 1-00:00:00      2   down nid[001371,002760]
scania       up 7-00:00:00     40   resv nid[001716-001755]
scania       up 7-00:00:00     16  alloc nid[001532-001547]
scania       up 7-00:00:00    168   idle nid[001548-001715]
scania-hf    up 7-00:00:00      4   idle nid[000011-000014]
memory       up 7-00:00:00      2 drain$ nid[001847,001910]
memory       up 7-00:00:00     10  maint nid[001844-001846,001848-001851,001908-001909,00191
memory       up 7-00:00:00    290  alloc nid[000101-000118,001512-001531,001756-001767,00177
919,001924-002023,002552-002567]
shared       up 7-00:00:00     40   mix- nid[001000-001005,001007-001010,001013,001015,00102
052-001054,001068-001069,001071,001074,002568-002572,002574-002576,002578-002579,002584,0025
shared       up 7-00:00:00      1 drain$ nid001036
shared       up 7-00:00:00      3  maint nid[001037-001039]
shared       up 7-00:00:00     32    mix nid[001006,001011,001014,001016,001021,001023,00102
056,001059-001063,001066,001070,001072-001073,002573,002580,002582-002583,002585-002586]
shared       up 7-00:00:00     20  alloc nid[001012,001017-001019,001026,001029,001031,00103
067,001075,002577,002581]
long         up 7-00:00:00      1 drain$ nid002676
long         up 7-00:00:00      3  maint nid[002677-002679]
long         up 7-00:00:00    124  alloc nid[001800-001819,002588-002675,002680-002695]
eggnog       up 7-00:00:00      2  alloc nid[002536-002537]
eggnog       up 7-00:00:00      2   idle nid[002538-002539]
supernova    up 14-00:00:0     12   idle nid[002540-002551]
```